UML DIAGRAMS

Digital Certificate Verification & Revocation Platform

ISSUANCE
Generate • Hash • Sign • QR

VERIFICATION
Public page • Integrity • Status

GOVERNANCE
Issuer • Revocation • Audit

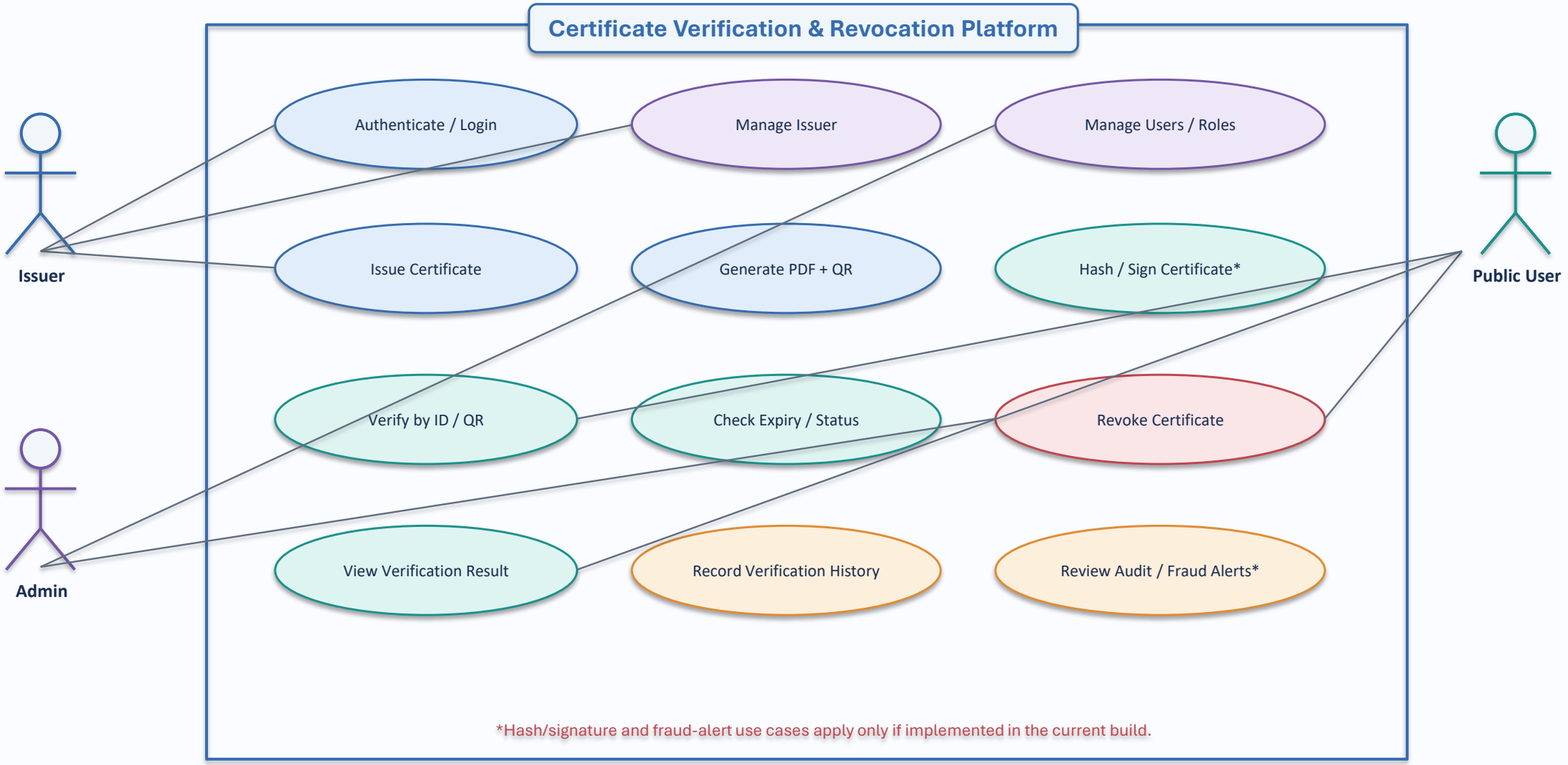
Professional, editable diagrams based on the stated project scope and previously shared module/technology details.

Design baseline: Next.js / React frontend • Java Spring Boot backend • PostgreSQL • JWT / RBAC

Note: Optional features are labeled as design assumptions; validate against the actual implementation before submission.

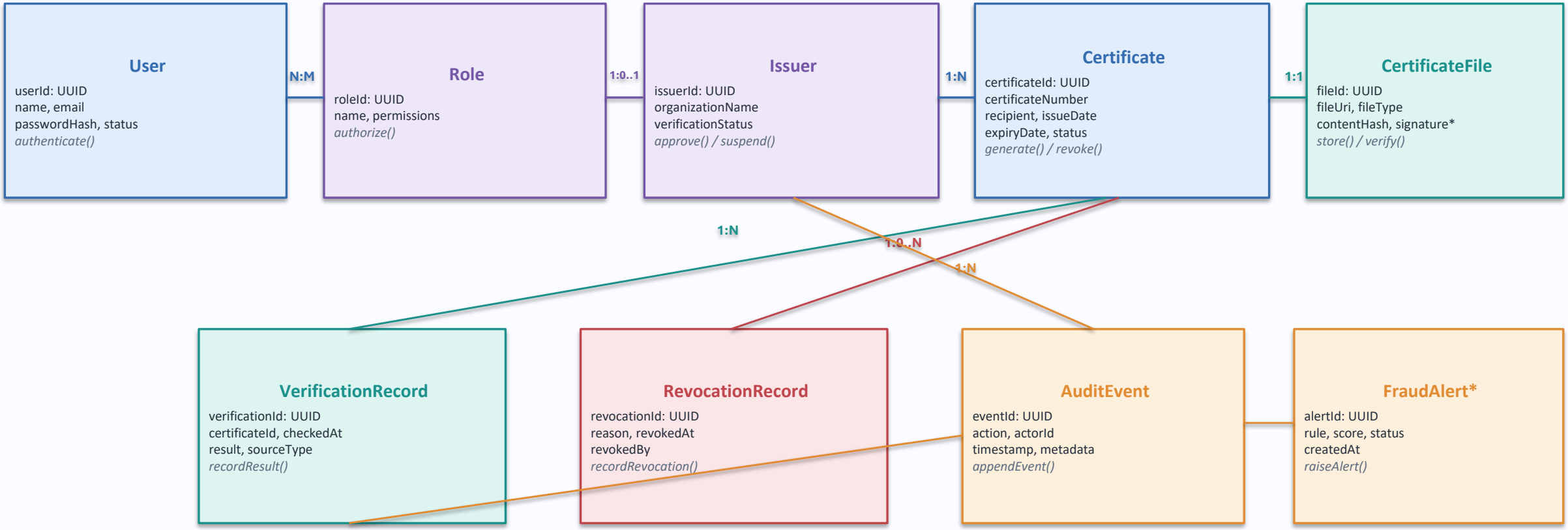
1. Use Case Diagram

Actors and main system functions; public verification is available without issuer login.



2. Domain Class Diagram

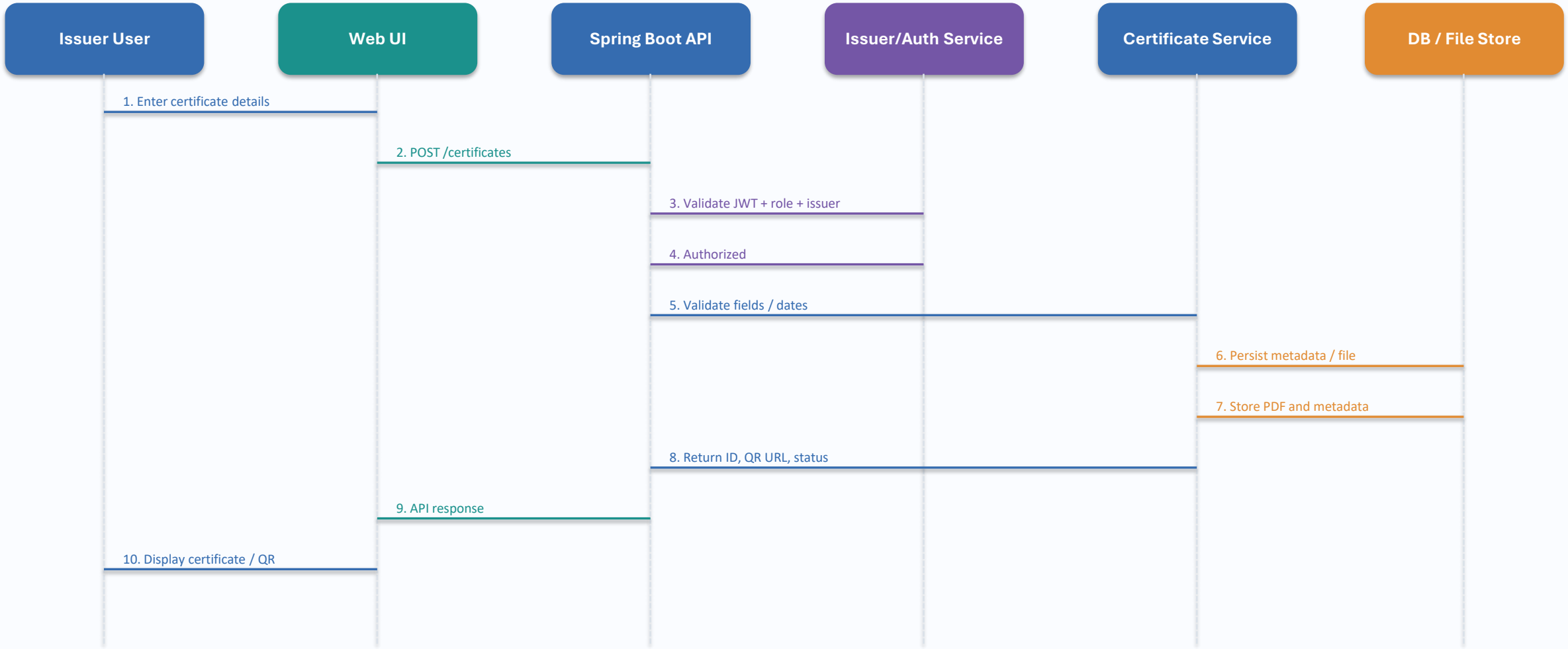
Core domain model; attributes are illustrative design-level fields and should be aligned with the final database schema.



Relationship multiplicities are proposed; confirm against actual entity mapping and foreign-key constraints. *Optional features.

3. Sequence Diagram — Certificate Issuance

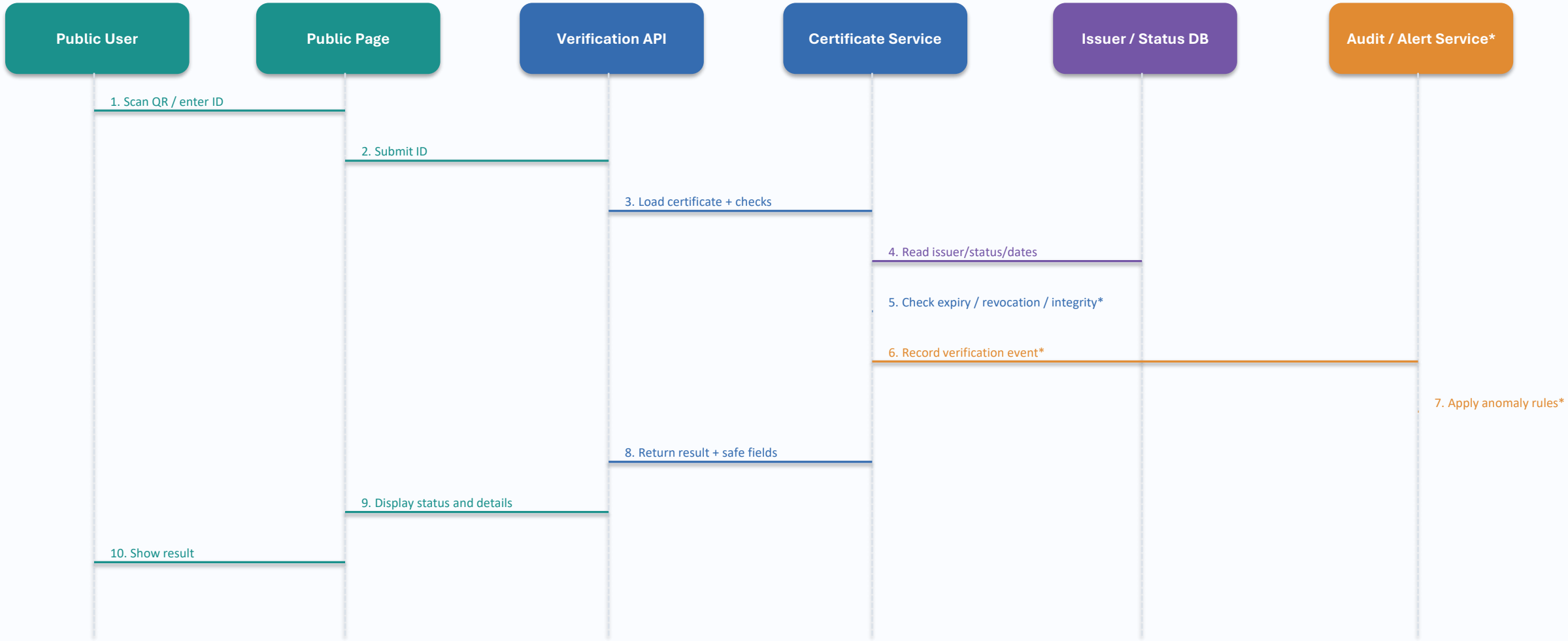
Successful issuance path for an authenticated, authorized issuer.



Security note: create the QR as a pointer to the official verification route; it must not itself be treated as proof of authenticity.

4. Sequence Diagram — Public Verification

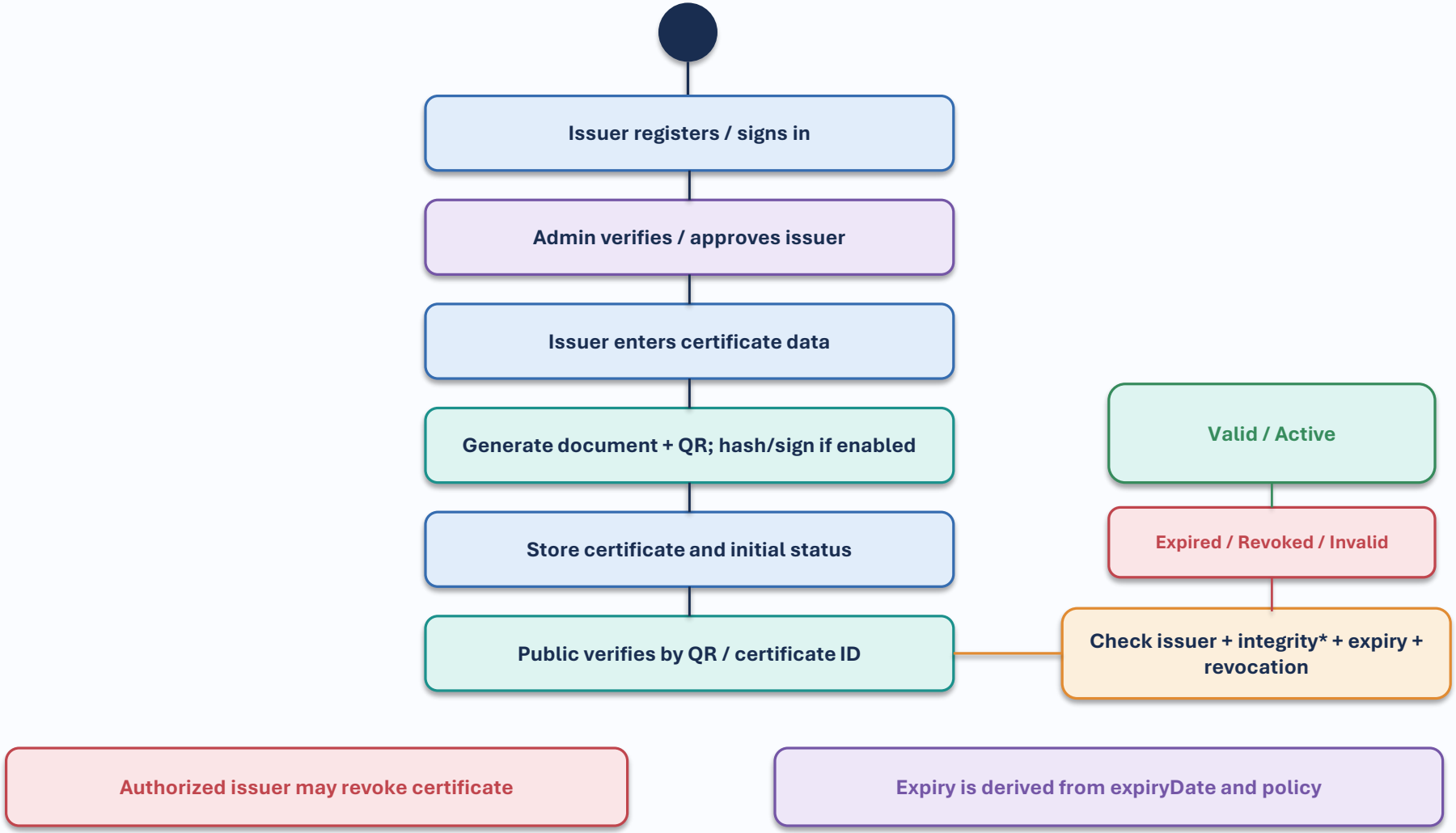
Public flow: QR/ID → server-side checks → status result → verification history.



*Integrity/signature checks, event recording and anomaly rules must be confirmed against the implemented code.

5. Activity Diagram — Certificate Lifecycle

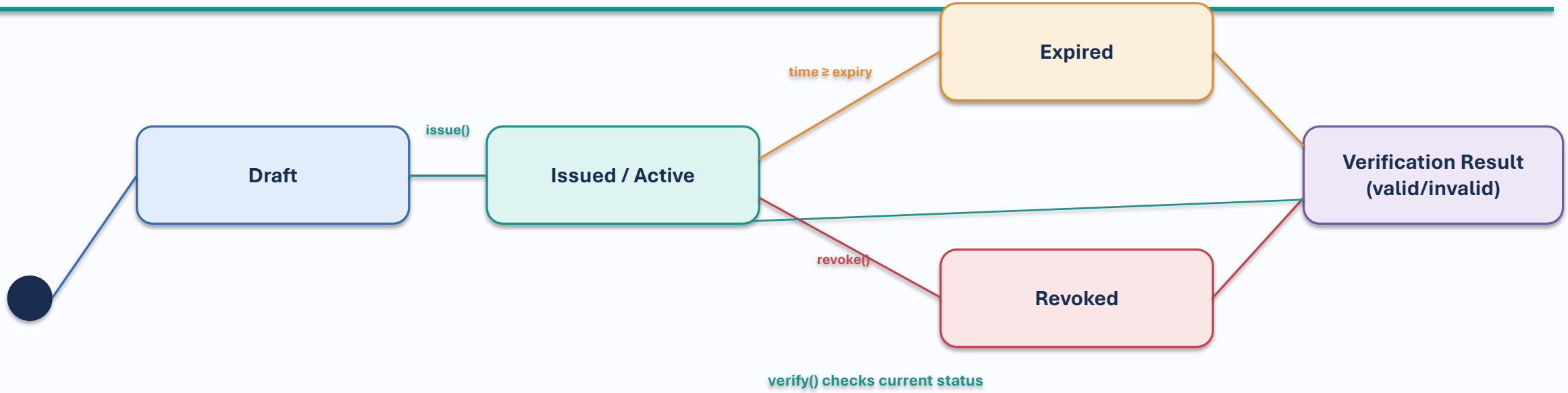
Main lifecycle from issuer onboarding through issuance, verification, expiry and revocation.



*Use only the integrity/signature mechanisms actually implemented.

6. State Machine Diagram — Certificate Status

Logical states; define exact transition rules and whether expiry is stored or computed.

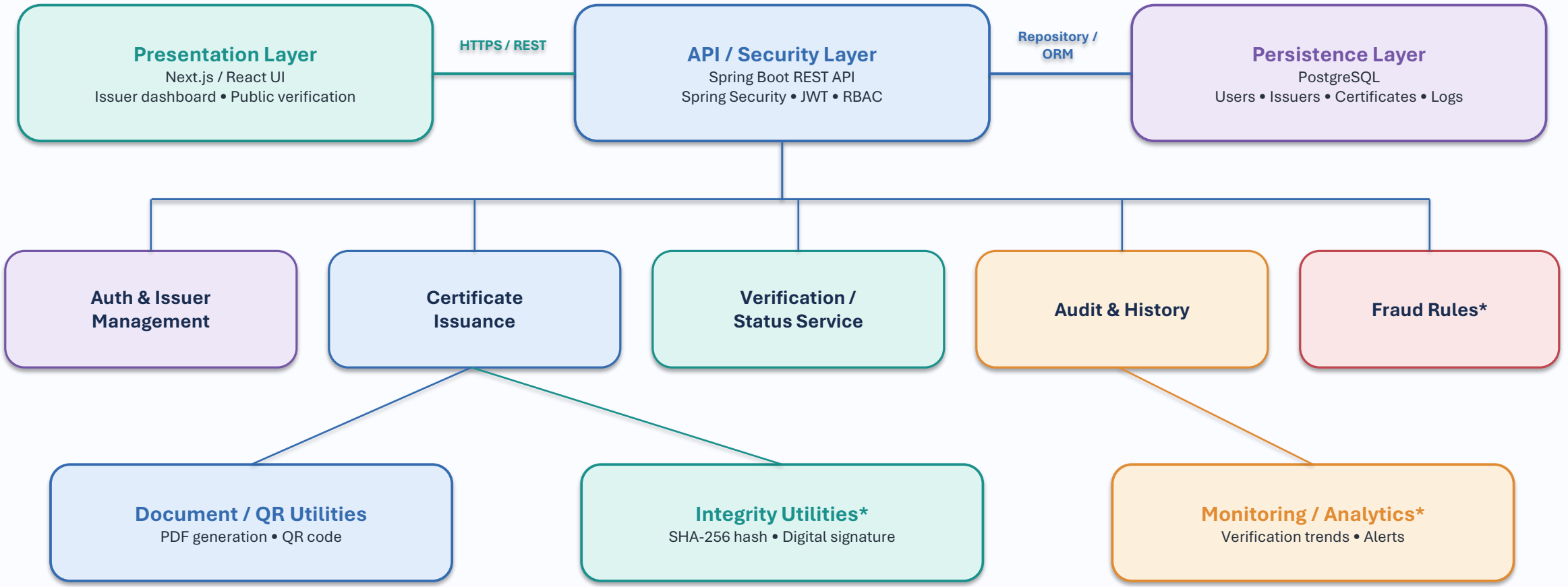


Design rule: a revoked certificate must never be displayed as active. Expiry may be computed at verification time or persisted by a scheduled process; document the actual choice.

A verification result is an outcome, not necessarily a permanent certificate lifecycle state.

7. Component Diagram — Logical Architecture

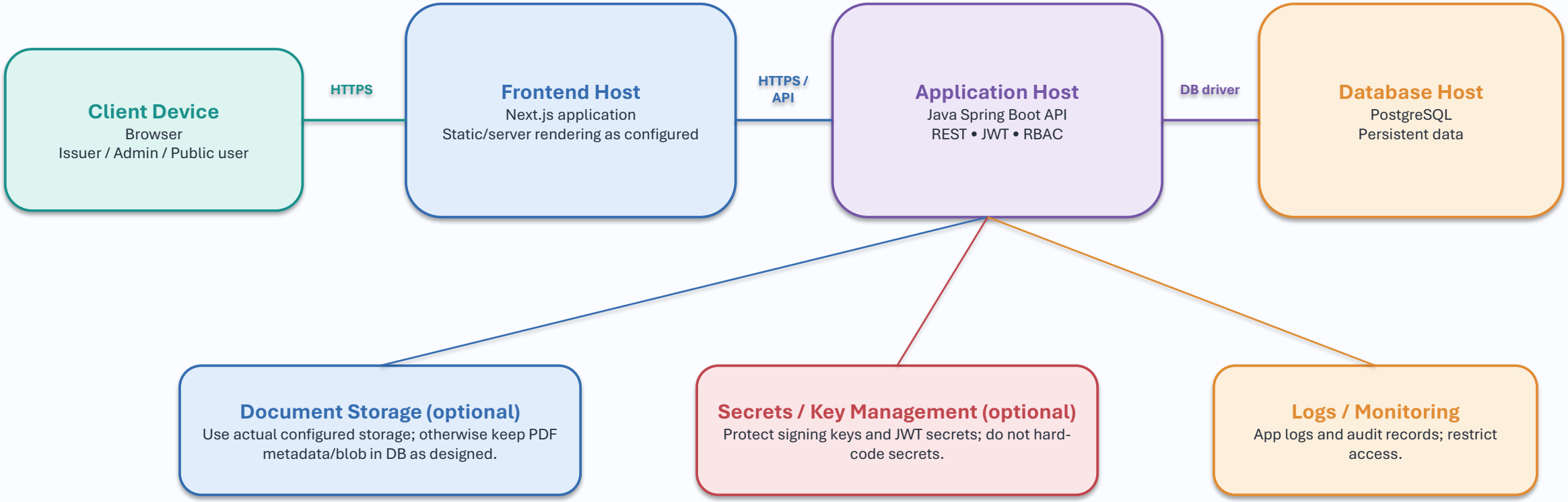
Maps to the stated Next.js/React frontend, Spring Boot services and PostgreSQL persistence.



*Hash/signature, rule engine and analytics are included only to the extent implemented; no blockchain component is assumed.

8. Deployment Diagram — Hosting View

Logical deployment baseline; replace with actual hosting providers and network topology before submission.



Deployment is conceptual until the team confirms actual hostnames, TLS setup, storage strategy, signing-key location and database access controls.

9. Design Constraints & Assumptions

Keep these notes with the diagrams to prevent accidental overclaiming in the submitted report.

QR is a pointer

QR should open the official verification endpoint; QR content alone is not proof of authenticity.

Hash vs digital signature

SHA-256 detects data changes only when compared with a trusted stored hash. A digital signature additionally requires protected signing keys and signature verification.

Expiry / revocation

Verification must check current lifecycle status, not only whether a QR exists.

Privacy

Public verification response should expose only approved fields; never return password hashes, internal tokens or private audit metadata.

Fraud detection

Rule-based anomaly alerts are not machine-learning accuracy claims; thresholds and test outcomes must be documented.

Blockchain

Blockchain is not assumed in the baseline design because the stated stack does not require it.

Before submission: align class attributes, endpoint names, database relations, and optional modules with the actual codebase.